

IT Expense System

Beginner From-Scratch Build Tutorial

Updated after backend connection: Angular frontend, Bootstrap styling, Node.js Express API, dummy data store, real request workflow, validation, attachments, dashboard counts, notifications, and future SQL Server integration.

Updated: June 24, 2026

Beginner promise: This guide explains what to build, why each folder exists, where each file belongs, how components connect to services and routes, how backend endpoints work, and how the current dummy store can later become SQL Server.

1. The Big Mental Model

The app has three layers. The Angular frontend is what the user sees. The Express backend is the API Angular calls. The dummy store is a temporary in-memory database replacement. Later, SQL Server will replace the dummy store while the frontend and route names can mostly stay the same.

```
User clicks button
-> Angular component handles click
-> Component calls feature service or store
-> ApiService sends HTTP request
-> Express middleware checks token and role
-> Express route validates status and fields
-> dummy-store.js reads or changes data
-> JSON returns to Angular
-> Component updates the UI
```

Piece	Beginner explanation
Component	A screen or part of a screen, such as RequestListPage or NotificationsPage.
Template HTML	The markup where forms, buttons, tables, Bootstrap classes, and Angular control flow live.
Service	Reusable TypeScript logic. Example: ApiService knows the backend base URL.
Store	Frontend store holds request state and calls RequestApiService. Backend dummy store holds temporary server data.
Route	Frontend routes show screens. Backend routes expose HTTP endpoints.
Middleware	Backend code that runs before endpoints, usually auth and role checks.
Validator	Backend code that rejects bad request data before it reaches the store.

2. Current Project Structure

Use this structure when rebuilding from scratch. It keeps responsibilities separate and makes the SQL Server change safer later.

Folder or file	Purpose
src/app/core/api	ApiService and ApiEndpoints. All frontend backend paths are centralized here.
src/app/core/auth	Login session, token interceptor, auth guard, role guard.
src/app/features/requests	Request list, detail, form, remarks modal, request API service, and request store.
src/app/features/dashboard	Backend-powered request dashboard counts and approval report.
src/app/features/notifications	Backend-powered notification list, unread count, mark read, mark all read.
src/app/shared/layout	Main sidebar layout and navigation.
backend/server.js	Express app entry point. Mounts all backend route files.
backend/routes	Express route files grouped by module.
backend/middleware	Backend auth and role middleware.
backend/validators	Backend validation before saving requests.
backend/store/dummy-store.js	Temporary in-memory data layer. Later becomes SQL Server repositories.
backend/uploads	Temporary local file storage for uploaded attachments.

3. Start From Zero

Create Angular app

Why: Angular CLI creates the frontend structure, TypeScript setup, routing, and build config.

```
ng new it-expense --routing --style=css
cd it-expense
```

Install dependencies

Why: Bootstrap styles the UI. Express is the API server. CORS allows Angular to call Express.

```
npm.cmd install bootstrap express cors
```

Create backend folder

Why: Keep backend code separate from Angular so it can grow into a real API.

```
mkdir backend
mkdir backend\routes backend\middleware backend\store backend\validators backend\data
```

Add scripts

Why: Scripts make startup repeatable for beginners.

```
"start": "ng serve",
"start:api": "node backend/server.js",
"dev:api": "node --watch backend/server.js"
```

4. Configure Bootstrap And Custom Styles

Bootstrap gives you ready-made form controls, alerts, buttons, cards, badges, tables, grids, and responsive spacing. Your custom app style belongs in global theme files for tokens and component CSS files for screen-specific polish.

```
/* angular.json styles */
node_modules/bootstrap/dist/css/bootstrap.min.css
src/theme.css
src/styles.css
```

- Use Bootstrap classes for layout and common components.
- Use src/theme.css for app colors, CSS variables, and reusable visual decisions.
- Use each component CSS file only for styles that belong to that component.
- Do not put all screen-specific CSS into one global file.

5. Build Authentication First

Authentication is the first backend connection because every protected request needs a token. In this app, the backend returns a simple dummy token. Angular stores it in localStorage and an interceptor adds it to API requests.

File	Role
backend/routes/auth.routes.js	POST /api/auth/login checks email and password against dummy users.
backend/middleware/auth.middleware.js	Reads Bearer token, finds the user, attaches req.user.
src/app/core/auth/auth.service.ts	Calls login endpoint and stores session.
src/app/core/auth/auth-token.interceptor.ts	Adds Authorization header to backend calls.
src/app/core/auth/auth.guard.ts	Blocks routes when not logged in.

File	Role
src/app/core/auth/role.guard.ts	Blocks routes when the logged-in user lacks the required role.

```
Frontend login flow
LoginPage.submit()
-> AuthService.login()
-> ApiService.post('/auth/login')
-> Express auth route
-> session saved
-> interceptor adds token to future requests
```

6. Build The Express Backend

The backend is intentionally modular. server.js should stay small. Each feature gets a route file. Middleware handles auth and role checks before the route handler does business work.

```
backend/server.js
app.use('/api/auth', authRoutes)
app.use('/api/expense-requests', requestRoutes)
app.use('/api/workflow', workflowRoutes)
app.use('/api/reports', reportRoutes)
app.use('/api', miscRoutes)
```

- Use express.json() so JSON request bodies are available as req.body.
- Use cors() so Angular can call the backend during local development.
- Use one fallback 404 handler for unknown API routes.
- Use one error handler for unexpected backend errors.

7. Request Backend Routes

Requests are the heart of the system. Angular calls these routes through RequestApiService. The backend checks token, role, status, and validation before changing data.

Endpoint	What it does
GET /api/expense-requests	List requests visible to the current role.
POST /api/expense-requests/drafts	Create a draft request.
POST /api/expense-requests/submit	Submit a new request or submit/resubmit an existing draft/returned request.
PATCH /api/expense-requests/:id/draft	Edit a draft or returned request.
DELETE /api/expense-requests/:id/draft	Delete a draft request.
GET /api/expense-requests/:id	Read request detail only if visible to current role.
POST /api/expense-requests/:id/cancel	Cancel a draft, returned, or pending endorsement request by owner/admin.

8. RequestStoreService And RequestApiService

The frontend was simplified into two responsibilities. RequestApiService only knows backend endpoints. RequestStoreService owns UI state, visible requests, local fallback, loading/error/success messages, and workflow methods.

Service	Responsibility
RequestApiService	Thin backend API wrapper: list, create, update, submit, workflow actions, attachments.

Service	Responsibility
RequestStoreService	State layer: signals, cached request list, loading/error/success messages, action guards.
ApiService	Generic HTTP wrapper with base URL and query/download helpers.

```

Component
-> store.submit(id)
-> RequestStoreService.runRequestAction()
-> RequestApiService.submit(id)
-> ApiService.post('/expense-requests/submit')

```

9. Workflow Status Rules

The frontend hides invalid buttons, but the backend must be the final authority. A user can fake an HTTP call, so Express must reject wrong-role and wrong-status actions.

Action	Allowed backend rule
Submit	Draft or Returned -> PendingEndorsement by original Requester/Admin.
Endorse	PendingEndorsement -> PendingApproval by Endorser/Admin.
Return	PendingEndorsement by Endorser or PendingApproval by Approver. Remarks required.
Reject	PendingEndorsement by Endorser or PendingApproval by Approver. Remarks required.
Approve	PendingApproval -> Approved by Approver/Admin.
Close	Approved -> Closed by Finance/Admin.
Terminal statuses	Rejected, Cancelled, and Closed cannot move forward.

10. Backend Validation

Frontend validation helps users, but backend validation protects the system. The backend now validates request fields before saving or updating data.

- Title, justification, department, cost center, request type, and line items are required.
- Department, cost center, and category must exist in the dummy reference lists.
- Request type must be Expense, Purchase, or Reimbursement.
- Line item quantity and unit amount must be positive.
- Seller links must be valid http or https URLs.
- The backend returns 400 with an errors array. The frontend shows those messages on the form.

```

backend/validators/request.validator.js
validateRequestCommand(req.body, store.state)
if (!validation.valid) return res.status(400).json({ message, errors })

```

11. Attachments Upload, View, And Download

The placeholder was replaced with a simple real file flow. Angular reads the selected file as Base64, sends it to Express, and Express writes the file under backend/uploads. Metadata stays in dummy-store.js for now.

Route	Purpose
GET /expense-requests/:requestId/line-items/:lineItemId/attachments	List files for a line item.
POST /expense-requests/:requestId/line-items/:lineItemId/attachments	Upload one file for a draft/returned request line item.
GET /files/:fileId/download	Stream file bytes to the browser for View or Download.

- Requester/Admin can upload while request is Draft or Returned.
- Visible roles can view/download files for requests they are allowed to see.
- Current max file size is 5 MB.
- When SQL Server is added, store metadata in SQL Server and keep file bytes in disk, cloud storage, or FILESTREAM depending on company policy.

12. Dashboard Counts From Backend

The dashboard is no longer a mock placeholder. It calls GET /api/reports/request-dashboard and displays counts based on the logged-in user's visible requests.

Count	Meaning
Total Visible	How many requests this role can currently see.
Pending Action	PendingEndorsement and PendingApproval visible to this role.
Urgent	Visible requests marked urgent.
Total Amount	Sum of visible request amounts.
Status Counts	Breakdown by Draft, Returned, PendingEndorsement, PendingApproval, Approved, Rejected, Closed, Cancelled.
Recent Visible Requests	Latest visible requests sorted by updatedAt.

13. Notifications

Notifications are generated by backend workflow events and shown on the Notifications page. They are stored in memory for now.

Event	Who gets notified
Request Submitted or Resubmitted	Endorsers.
Request Returned	Original Requester.
Request Approved	Original Requester and Finance users.

```
GET /api/notifications
GET /api/notifications/unread-count
PATCH /api/notifications/:id/read
PATCH /api/notifications/read-all
```

14. Loading, Error, And Success Messages

Backend calls are asynchronous. The UI should tell users what is happening, prevent duplicate clicks, and show errors clearly. RequestStoreService owns this behavior for request actions.

- loadingMessage appears while backend calls run.

- successMessage confirms the exact workflow result.
- errorMessage shows backend validation and permission errors.
- pendingOperationKeys prevents duplicate backend actions.
- Angular untracked() prevents refresh effects from re-triggering loading forever.

15. Manual Test Checklist

Area	What to test
Login	Use requester@test.com, endorser@test.com, approver@test.com, finance@test.com, admin@test.com. Password is password123.
Create draft	Requester creates request and saves as draft. Draft appears in request list.
Edit draft	Requester opens draft detail, edits, adds/removes line items, uploads attachment.
Submit	Requester submits. Endorser receives notification and sees request in queue.
Return	Endorser returns with remarks. Requester sees returned request, remarks, and notification.
Resubmit	Requester edits returned request and submits again.
Endorse	Endorser endorses. Approver sees request.
Approve	Approver approves. Requester and Finance receive notification.
Close	Finance closes approved request.
Dashboard	Each role sees backend counts for visible requests.

16. Troubleshooting

Problem	Likely fix
Route not found	Restart backend. Express loads route files only when server starts.
Frontend shows old UI	Refresh Angular browser tab or restart npm start.
401 Authentication is required	Login again. Token may be missing from localStorage.
403 Permission error	You are using the wrong role for that action.
409 Status error	The request is not in the correct workflow status for that action.
Upload button missing	Only requester/admin can upload while request is Draft or Returned.
Uploaded file lost after restart	Metadata is still in memory. SQL Server persistence is Phase 2.

17. SQL Server Migration Plan

Do not connect SQL Server by rewriting the whole app. Replace the backend store gradually with repositories. Keep route names and frontend services stable.

```
backend
db
sql-server.js
repositories
users.repository.js
requests.repository.js
workflow.repository.js
```

```
attachments.repository.js
notifications.repository.js
routes
requests.routes.js
```

- Create SQL tables for Users, Requests, LineItems, Attachments, AuditLogs, Notifications.
- Move dummy-store functions into repository files one module at a time.
- Keep validation in backend/validators before repository calls.
- Keep middleware auth and role checks before route handlers.
- Add transactions for workflow actions so status, audit log, and notification save together.
- Use parameterized SQL queries. Never build SQL with string concatenation from user input.

18. Suggested Learning Resources

Use these after reading the guide. Start with Angular and Express, then Bootstrap styling, then SQL Server connectivity.

Topic	Resource
Angular basics	https://angular.dev/tutorials
Express install and routing	https://expressjs.com/en/starter/installing/
Bootstrap components and customization	https://getbootstrap.com/docs/5.3/getting-started/introduction/
Node.js driver for SQL Server	https://learn.microsoft.com/en-us/sql/connect/node-js/node-js-driver-for-sql-server?view=sql-server-ver17

19. Beginner Build Order Summary

1. Generate Angular app.
2. Install Bootstrap, Express, and CORS.
3. Add frontend core folders: api, auth, models, mock.
4. Add shared layout and navigation.
5. Build login and route guards.
6. Add backend server, auth route, middleware, and dummy data.
7. Build request list, form, detail, and workflow actions.
8. Connect requests to backend through RequestApiService.
9. Add loading, error, success messages, and duplicate-click protection.
10. Add audit trail, remarks, editable drafts, returned resubmission.
11. Add attachments, backend validation, strict workflow rules.
12. Add dashboard counts and notifications.
13. Replace dummy-store with SQL Server repositories when ready.